Lesson 40: Finite-Horizon MDPs

AI/ML Mathematics

Contents

1 Motivation and Overview 2

2 References 2

3 Finite-Horizon MDP Formulation 3

4 The Principle of Optimality and Bellman Equations 4

5 Backward Induction 5

6 Worked Examples 5

7 Finite-Horizon Bellman Expectation Equations 7

8 Exercises 7

9 Summary 8

10 Further Reading 9

1 Motivation and Overview

Consider a Mars rover that must plan a sequence of scientific measurements over a mission of exactly $T = 30$ days. Each day the rover observes its current location and instrument status (state), selects a measurement action (drill, photograph, traverse, charge battery), and receives a scientific value reward. The mission has a fixed terminal time: after day 30, no further actions are possible. Should the rover adopt the same strategy on day 1 as on day 29? On the last day it should exploit the best remaining measurement, while on early days it may invest in traversal to reach high-value locations. The finite horizon fundamentally changes the structure of optimal decision-making – the best action depends not only on the current state but also on how much time remains.

The finite-horizon MDP formulation captures this temporal structure. Its analysis via backward induction yields the first practical algorithm for computing optimal policies, and the principle of optimality that emerges underpins all of dynamic programming.

Why Finite-Horizon MDPs Matter for RL/ML

- **Episodic RL**: Most RL benchmarks (CartPole, Atari, board games) are episodic with a finite horizon; the finite-horizon MDP is their theoretical model.
- **Model-based planning**: Backward induction is the prototype for model-based methods such as MuZero's tree search.
- **Batch RL and offline planning**: Finite-horizon problems arise in clinical trials, inventory management, and resource allocation where the planning horizon is fixed.
- **Foundation for infinite-horizon theory**: The infinite-horizon discounted MDP (Lesson 41) is derived by taking $T \to \infty$; understanding the finite case clarifies the limiting argument.

Prerequisites. MDP model, policy classes (history-dependent, Markov, stationary, deterministic), return, state-value function, action-value function, and the Bellman expectation equations (Lesson 39). Conditional expectation and the tower property (Lessons 9–12). Finite induction and basic proof techniques.

Learning objectives. After completing this lesson, the student will be able to:

- *Define* a finite-horizon MDP and its time-dependent value functions.
- *Derive* the finite-horizon Bellman optimality equations via the principle of optimality.
- *Prove* the existence of a deterministic Markov optimal policy for finite-horizon MDPs.
- *Apply* backward induction to compute optimal policies for small examples.
- *Analyze* the computational complexity of backward induction.
- *Compare* finite-horizon and infinite-horizon MDP formulations and identify when each applies.

2 References

- Puterman [2014, Chapter 4] – Comprehensive treatment of finite-horizon MDPs, optimality equations, and backward induction.
- Bertsekas [2012, Chapter 1, Sections 1.2–1.3] – Dynamic programming and the principle of optimality, with finite-horizon formulation.

- Sutton and Barto [2018, Chapter 3] – Episodic tasks and returns with finite horizons.
- Howard [1960] – Historical introduction of dynamic programming for Markov processes.
- Bertsekas and Tsitsiklis [1996] – Neuro-dynamic programming treatment of finite-horizon problems and approximate methods.

3 Finite-Horizon MDP Formulation

Definition 3.1 (Finite-Horizon MDP)**.** A *finite-horizon MDP* is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, T)$ where:

1. $\mathcal{S}$ is a finite state space with $|\mathcal{S}| = S$.
2. $\mathcal{A}$ is a finite action space with $|\mathcal{A}| = A$.
3. $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition kernel (possibly time-dependent: $P_t(s' \mid s, a)$).
4. $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function (possibly time-dependent: $R_t(s, a)$), with an optional terminal reward $R_T : \mathcal{S} \to \mathbb{R}$.
5. $T \in \mathbb{N}$ is the *horizon* (number of decision stages).

The agent acts at stages $t = 0, 1, \ldots, T-1$ and the process terminates at stage T.

Remark 3.2. When $P_t = P$ and $R_t = R$ for all t, the finite-horizon MDP is *time-homogeneous*. Despite time-homogeneous dynamics, the optimal policy is generally time-dependent because the number of remaining stages affects the optimal action.

Definition 3.3 (Finite-Horizon Return)**.** The *finite-horizon return* from stage t is

$$G_t = \sum_{k=0}^{T-1-t} R_{t+k}(S_{t+k}, A_{t+k}) + R_T(S_T),$$

where $R_T(S_T)$ is the terminal reward. When no discounting is used (as is typical for finite-horizon problems), the return is the total undiscounted reward. A discounted variant uses $G_t = \sum_{k=0}^{T-1-t} \gamma^k R_{t+k}(S_{t+k}, A_{t+k}) + \gamma^{T-t} R_T(S_T)$.

Definition 3.4 (Time-Dependent Markov Policy)**.** A *time-dependent Markov policy* for a finite-horizon MDP is a sequence $\pi = (\pi_0, \pi_1, \ldots, \pi_{T-1})$ where each $\pi_t : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to action distributions. A *time-dependent deterministic policy* is a sequence $d = (d_0, d_1, \ldots, d_{T-1})$ where each $d_t : \mathcal{S} \to \mathcal{A}$. The set of all such policies is Π_T^{MD} and Π_T^{DD}, respectively.

Definition 3.5 (Finite-Horizon Value Functions)**.** For a time-dependent Markov policy π, the *value function at stage* t is

$$V_t^\pi(s) = \mathbb{E}_\pi[\, G_t \mid S_t = s \,] = \mathbb{E}_\pi\left[\sum_{k=0}^{T-1-t} R_{t+k}(S_{t+k}, A_{t+k}) + R_T(S_T) \;\middle|\; S_t = s \right].$$

The *optimal value function at stage* t is $V_t^*(s) = \sup_{\pi \in \Pi_T^{\mathrm{MD}}} V_t^\pi(s)$. Similarly, the *action-value function at stage* t is

$$Q_t^\pi(s, a) = \mathbb{E}_\pi[\, G_t \mid S_t = s,\, A_t = a \,].$$

Finite-Horizon Value Functions Are Time-Dependent

Unlike the infinite-horizon discounted setting where $V^\pi(s)$ is time-independent under a stationary policy, the finite-horizon value $V_t^\pi(s)$ depends on t even for time-homogeneous dynamics. The value of being in state s with 20 steps remaining is generally different from being in state s with 2 steps remaining. Failing to index value functions by time is a common source of errors.

4 The Principle of Optimality and Bellman Equations

The principle of optimality, due to Bellman, is the foundational insight that enables dynamic programming.

Theorem 4.1 (Principle of Optimality). *Let $\pi^* = (d_0^*, d_1^*, \ldots, d_{T-1}^*)$ be an optimal policy for the finite-horizon MDP. Then for any stage t and state s, the "tail policy" $(d_t^*, d_{t+1}^*, \ldots, d_{T-1}^*)$ is optimal for the sub-problem starting at stage t in state s. Formally,*

$$V_t^*(s) = \mathbb{E}_{\pi^*}[\, G_t \mid S_t = s \,] = \mathbb{E}_{(d_t^*, \ldots, d_{T-1}^*)}[\, G_t \mid S_t = s \,].$$

Proof. Suppose for contradiction that the tail policy $(d_t^*, \ldots, d_{T-1}^*)$ is not optimal from stage t at some state $\bar{s}$. Then there exists a policy $\hat{\pi} = (\hat{d}_t, \ldots, \hat{d}_{T-1})$ with $V_t^{\hat{\pi}}(\bar{s}) > V_t^{(d_t^*, \ldots, d_{T-1}^*)}(\bar{s})$. Define a new full policy $\tilde{\pi}$ by

$$\tilde{\pi} = (d_0^*, \ldots, d_{t-1}^*, \hat{d}_t, \ldots, \hat{d}_{T-1}).$$

By the Markov property, policies before stage t affect only the probability of reaching $\bar{s}$ at stage t, not the conditional value from $\bar{s}$ onward. Therefore, $V_0^{\tilde{\pi}}(s_0) \geq V_0^{\pi^*}(s_0)$ with strict inequality when there is positive probability of reaching $\bar{s}$ at stage t. This contradicts the optimality of π^*. □

Theorem 4.2 (Finite-Horizon Bellman Optimality Equations). *The optimal value functions satisfy the backward recursion:*

1. ***Boundary condition:*** *$V_T^*(s) = R_T(s)$ for all $s \in \mathcal{S}$.*
2. ***Recursion:*** *For $t = T-1, T-2, \ldots, 0$ and all $s \in \mathcal{S}$,*

$$V_t^*(s) = \max_{a \in \mathcal{A}} \left[R_t(s,a) + \sum_{s' \in \mathcal{S}} P_t(s' \mid s,a)\, V_{t+1}^*(s') \right]. \tag{1}$$

Proof. We prove this by backward induction on t.

Base case ($t = T$): At the terminal stage no actions are taken, so $V_T^*(s) = R_T(s)$ by definition.

Inductive step: Assume V_{t+1}^* satisfies the optimality equation. For any policy π and state s,

$$\begin{aligned} V_t^\pi(s) &= \mathbb{E}_\pi[R_t(S_t, A_t) + G_{t+1} \mid S_t = s] \\ &= \sum_a \pi_t(a \mid s) \left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^{\pi^{(t+1)}}(s') \right], \end{aligned}$$

where $\pi^{(t+1)} = (\pi_{t+1}, \ldots, \pi_{T-1})$ is the tail policy. By the inductive hypothesis, $V_{t+1}^{\pi^{(t+1)}}(s') \leq V_{t+1}^*(s')$ for all s' and all tail policies. Therefore,

$$\begin{aligned} V_t^\pi(s) &\leq \sum_a \pi_t(a \mid s) \left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^*(s') \right] \\ &\leq \max_{a \in \mathcal{A}} \left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^*(s') \right], \end{aligned}$$

where the second inequality uses $\sum_a \pi_t(a \mid s) f(a) \leq \max_a f(a)$. This upper bound is achieved by the deterministic policy $d_t^*(s) = \arg\max_{a \in \mathcal{A}} [R_t(s,a) + \sum_{s'} P_t(s' \mid s,a) V_{t+1}^*(s')]$, which verifies (1). □

Finite-Horizon Bellman Optimality Recursion

The optimal value functions for a finite-horizon MDP satisfy:

$$V_T^*(s) = R_T(s),$$
$$V_t^*(s) = \max_{a\in\mathcal{A}} \left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^*(s') \right], \quad t = T-1, \dots, 0.$$

The optimal policy is obtained by recording the maximizing action at each stage and state: $d_t^*(s) = \arg\max_a [R_t(s,a) + \sum_{s'} P_t(s' \mid s,a) V_{t+1}^*(s')]$.

5 Backward Induction

Backward induction is the algorithmic realization of the Bellman optimality recursion. It computes the optimal value function and policy by sweeping backward from the terminal stage.

Definition 5.1 (Backward Induction Algorithm)**.** Given a finite-horizon MDP $(\mathcal{S}, \mathcal{A}, P, R, T)$:

1. **Initialize:** Set $V_T^*(s) = R_T(s)$ for all $s \in \mathcal{S}$.
2. **Backward sweep:** For $t = T-1, T-2, \dots, 0$:
   (a) For each $s \in \mathcal{S}$, compute $Q_t^*(s,a) = R_t(s,a) + \sum_{s'} P_t(s' \mid s,a) V_{t+1}^*(s')$ for all $a \in \mathcal{A}$.
   (b) Set $V_t^*(s) = \max_a Q_t^*(s,a)$.
   (c) Record $d_t^*(s) = \arg\max_a Q_t^*(s,a)$.
3. **Output:** Optimal policy $\pi^* = (d_0^*, d_1^*, \dots, d_{T-1}^*)$ and value functions $V_0^*, V_1^*, \dots, V_T^*$.

Theorem 5.2 (Existence of Deterministic Markov Optimal Policy)**.** *For any finite-horizon MDP with finite $\mathcal{S}$ and $\mathcal{A}$, there exists a deterministic Markov policy $\pi^* = (d_0^*, \dots, d_{T-1}^*) \in \Pi_T^{\mathrm{DD}}$ that is optimal. Moreover, backward induction computes such a policy.*

Proof. The proof follows directly from the backward induction in Theorem 4.2. At each stage t, the maximizer $d_t^*(s) = \arg\max_a [R_t(s,a) + \sum_{s'} P_t(s' \mid s,a) V_{t+1}^*(s')]$ exists because $\mathcal{A}$ is finite (the maximum of a finite set of real numbers is always attained). The resulting policy $\pi^* = (d_0^*, \dots, d_{T-1}^*)$ is deterministic (a single action per state-time pair) and Markov (depends only on the current state, not the full history). By Theorem 4.2, $V_t^{\pi^*}(s) = V_t^*(s)$ for all t and s. □

Remark 5.3. The optimal finite-horizon policy is generally *non-stationary*: $d_t^* \neq d_{t'}^*$ for $t \neq t'$. This is because the optimal action depends on the time remaining $T - t$, not just the current state. An agent that follows the same rule at every stage may be highly suboptimal.

Proposition 5.4 (Complexity of Backward Induction)**.** *Backward induction computes the optimal policy and value function for a finite-horizon MDP in $O(S^2AT)$ arithmetic operations and $O(ST)$ space (or $O(S)$ space if only the policy is stored and values are overwritten at each stage).*

Proof. At each stage t and state s, computing $Q_t^*(s,a)$ for a single action a requires summing over S successor states: $\sum_{s'} P_t(s' \mid s,a) V_{t+1}^*(s')$ costs $O(S)$ operations. Computing this for all A actions costs $O(SA)$, and finding the maximum over A actions costs $O(A)$. Repeating for all S states costs $O(S^2A)$ per stage. Over T stages, the total cost is $O(S^2AT)$. Storing V_t^* for all stages requires $O(ST)$ space; if only V_{t+1}^* is needed for the current computation, $O(S)$ value storage suffices (the policy d_t^* still requires $O(ST)$ storage). □

6 Worked Examples

Example 6.1 (Shortest Path on a Grid)**.** Consider a 2×3 grid with states $\mathcal{S} = \{(i,j) : i \in \{1,2\}, j \in \{1,2,3\}\}$ and actions $\mathcal{A} = \{\text{Right (R)}, \text{Up (U)}\}$. The agent starts at $(1,1)$ and must

reach $(2,3)$ within $T = 3$ steps. Transitions are deterministic: R moves $(i,j) \to (i,j+1)$ if $j < 3$, otherwise stay; U moves $(i,j) \to (i+1,j)$ if $i < 2$, otherwise stay. The reward is $R_t(s,a) = 0$ for all transitions and $R_T(s) = 10 \cdot \mathbf{1}\{s = (2,3)\}$.

Backward induction:

Stage $T = 3$: $V_3^*(s) = R_3(s)$, so $V_3^*(2,3) = 10$ and $V_3^*(s) = 0$ for all other states.

Stage $t = 2$: For each state s and action a, $Q_2^*(s,a) = 0 + V_3^*(s')$ where s' is the deterministic successor:

s	$Q_2^*(s,\mathrm{R})$	$Q_2^*(s,\mathrm{U})$	$V_2^*(s)$
$(1,1)$	$V_3^*(1,2) = 0$	$V_3^*(2,1) = 0$	0
$(1,2)$	$V_3^*(1,3) = 0$	$V_3^*(2,2) = 0$	0
$(1,3)$	$V_3^*(1,3) = 0$	$V_3^*(2,3) = 10$	10
$(2,1)$	$V_3^*(2,2) = 0$	$V_3^*(2,1) = 0$	0
$(2,2)$	$V_3^*(2,3) = 10$	$V_3^*(2,2) = 0$	10
$(2,3)$	$V_3^*(2,3) = 10$	$V_3^*(2,3) = 10$	10

Optimal actions at $t = 2$: $d_2^*(1,3) = \mathrm{U}$, $d_2^*(2,2) = \mathrm{R}$.

Stage $t = 1$:

s	$Q_1^*(s,\mathrm{R})$	$Q_1^*(s,\mathrm{U})$	$V_1^*(s)$
$(1,1)$	$V_2^*(1,2) = 0$	$V_2^*(2,1) = 0$	0
$(1,2)$	$V_2^*(1,3) = 10$	$V_2^*(2,2) = 10$	10
$(1,3)$	$V_2^*(1,3) = 10$	$V_2^*(2,3) = 10$	10
$(2,1)$	$V_2^*(2,2) = 10$	$V_2^*(2,1) = 0$	10
$(2,2)$	$V_2^*(2,3) = 10$	$V_2^*(2,2) = 10$	10
$(2,3)$	$V_2^*(2,3) = 10$	$V_2^*(2,3) = 10$	10

Stage $t = 0$:

s	$Q_0^*(s,\mathrm{R})$	$Q_0^*(s,\mathrm{U})$	$V_0^*(s)$
$(1,1)$	$V_1^*(1,2) = 10$	$V_1^*(2,1) = 10$	10

From $(1,1)$, the agent reaches $(2,3)$ in exactly 3 steps via either R-R-U, R-U-R, or U-R-R, all achieving reward 10. The optimal value is $V_0^*(1,1) = 10$.

Example 6.2 (Optimal Stopping Problem)**.** A gambler receives i.i.d. offers $X_1, X_2, \ldots, X_T$ drawn uniformly from $\{1, 2, \ldots, N\}$ with $N = 10$ and $T = 3$. At each stage t, the gambler observes X_t and either accepts (stops, receiving reward X_t) or rejects (continues to the next offer). If no offer is accepted by stage T, the gambler receives X_T automatically.

We model this as a finite-horizon MDP. The state at stage t is $s = X_t \in \{1, \ldots, 10\}$ (the current offer). Actions are $\mathcal{A} = \{\mathrm{Accept}, \mathrm{Reject}\}$. Accepting in state s yields terminal reward s and transitions to an absorbing state s_{done} with $R_T(s_{\mathrm{done}}) = 0$. Rejecting transitions to a new random state $s' \sim \mathrm{Uniform}\{1, \ldots, 10\}$ with zero intermediate reward. At stage T, the gambler must accept: $R_T(s) = s$.

Backward induction:

$V_3^*(s) = s$ for all s.

Stage $t = 2$: $Q_2^*(s, \mathrm{Accept}) = s$; $Q_2^*(s, \mathrm{Reject}) = \frac{1}{10}\sum_{s'=1}^{10} V_3^*(s') = \frac{1}{10} \cdot 55 = 5.5$. So $d_2^*(s) = \mathrm{Accept}$ if $s \geq 6$ and Reject if $s \leq 5$. $V_2^*(s) = \max(s, 5.5)$.

Stage $t = 1$: $Q_1^*(s, \mathrm{Reject}) = \frac{1}{10}\sum_{s'=1}^{10} V_2^*(s') = \frac{1}{10}(5 \times 5.5 + 6 + 7 + 8 + 9 + 10) = \frac{1}{10}(27.5 + 40) = 6.75$. So $d_1^*(s) = \mathrm{Accept}$ if $s \geq 7$ and Reject if $s \leq 6$. The threshold increases at earlier stages because the option to wait is more valuable.

Stage $t = 0$: $Q_0^*(s, \text{Reject}) = \frac{1}{10}\sum_{s'=1}^{10} V_1^*(s') = \frac{1}{10}(6\times 6.75+7+8+9+10) = \frac{1}{10}(40.5+34) = 7.45$. So $d_0^*(s) = \text{Accept}$ if $s \geq 8$ and Reject if $s \leq 7$.

The optimal policy is a threshold rule with increasing thresholds as time remaining grows: thresholds are $8, 7, 6$ for stages $t = 0, 1, 2$. This illustrates the non-stationary nature of optimal finite-horizon policies.

Optimal Stopping and Real Applications

Optimal stopping problems arise frequently in practice: deciding when to stop training a neural network (balancing training cost against validation performance), when to deploy a model, or when to accept a bid in an auction. The backward induction solution reveals that optimal thresholds decrease as the deadline approaches – a principle directly applicable to early stopping strategies in deep learning.

7 Finite-Horizon Bellman Expectation Equations

For policy evaluation (computing the value of a given policy rather than the optimal value), we need the finite-horizon analogue of the Bellman expectation equations.

Theorem 7.1 (Finite-Horizon Bellman Expectation Equations)**.** *For a time-dependent Markov policy $\pi = (\pi_0, \ldots, \pi_{T-1})$, the value functions satisfy:*

1. $V_T^\pi(s) = R_T(s)$.
2. *For $t = T-1, \ldots, 0$:*

$$V_t^\pi(s) = \sum_{a\in\mathcal{A}} \pi_t(a \mid s)\left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^\pi(s')\right]. \tag{2}$$

Proof. The boundary condition is immediate: at stage T, no actions remain, so $V_T^\pi(s) = R_T(s)$. For the recursion, using the return decomposition $G_t = R_t(S_t, A_t) + G_{t+1}$:

$$\begin{aligned}
V_t^\pi(s) &= \mathbb{E}_\pi[R_t(S_t, A_t) + G_{t+1} \mid S_t = s] \\
&= \sum_a \pi_t(a \mid s)\mathbb{E}_\pi[R_t(s,a) + G_{t+1} \mid S_t = s, A_t = a] \\
&= \sum_a \pi_t(a \mid s)\left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\,\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']\right] \\
&= \sum_a \pi_t(a \mid s)\left[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)\, V_{t+1}^\pi(s')\right].
\end{aligned}$$

□

Remark 7.2. In matrix form, defining P_t^π and r_t^π as in Lesson 39 (but now indexed by t), the backward recursion becomes $v_T^\pi = r_T$ and $v_t^\pi = r_t^\pi + P_t^\pi v_{t+1}^\pi$ for $t = T-1, \ldots, 0$. Unrolling: $v_0^\pi = \sum_{t=0}^{T-1}(\prod_{k=0}^{t-1} P_k^\pi) r_t^\pi + (\prod_{k=0}^{T-1} P_k^\pi) r_T$.

8 Exercises

Exercise 8.1 (Backward Induction Practice)**.** $(*)$
Consider a finite-horizon MDP with $\mathcal{S} = \{s_1, s_2\}$, $\mathcal{A} = \{a_1, a_2\}$, $T = 2$, and time-homogeneous dynamics: $P(s_1 \mid s_1, a_1) = 0.7$, $P(s_1 \mid s_1, a_2) = 0.3$, $P(s_1 \mid s_2, a_1) = 0.4$, $P(s_1 \mid s_2, a_2) = 0.9$. Rewards: $R(s_1, a_1) = 2$, $R(s_1, a_2) = 1$, $R(s_2, a_1) = 0$, $R(s_2, a_2) = 3$. Terminal reward: $R_2(s_1) = 5$, $R_2(s_2) = 0$. Apply backward induction to find $V_0^*(s)$ and $d_0^*(s)$ for both states.

Exercise 8.2 (Optimal Stopping Generalization). $(*)$
Generalize Example 6.2 to horizon $T = 4$ with offers drawn from $\{1, \ldots, 10\}$. Compute the optimal threshold at each stage and the expected value of the optimal policy.

Exercise 8.3 (Non-Stationarity of Optimal Policies). $(**)$
Consider a time-homogeneous finite-horizon MDP with $\mathcal{S} = \{A, B, C\}$, $\mathcal{A} = \{\text{left}, \text{right}\}$, $T = 4$, deterministic transitions:

$$\begin{aligned} &\text{left from } A \to B, \quad \text{right from } A \to C, \\ &\text{left from } B \to A, \quad \text{right from } B \to C, \\ &\text{left from } C \to C, \quad \text{right from } C \to C. \end{aligned}$$

Rewards: $R(A, \cdot) = 0$, $R(B, \cdot) = 1$, $R(C, \cdot) = 2$, $R_4(s) = 0$.

(a) Compute $V_t^*(s)$ for all t and s.

(b) Show that the optimal action at state A differs between $t = 0$ and $t = 3$, demonstrating non-stationarity.

(c) Compare with the infinite-horizon discounted optimal policy for $\gamma = 0.9$ (note: C is absorbing, so $V^*(C) = R(C, \cdot)/(1 - \gamma) = 20$).

Exercise 8.4 (Complexity Lower Bound). $(**)$
Prove that any algorithm computing the optimal value function $V_0^*(s)$ for all s in a finite-horizon MDP must read at least $\Omega(SAT)$ entries of the transition kernel in the worst case.

Hint: Construct two MDPs that differ in a single entry $P_t(s' \mid s, a)$ and show that the optimal values differ.

Exercise 8.5 (Finite Horizon to Infinite Horizon). $(***)$
Consider a time-homogeneous finite-horizon MDP with discount factor $\gamma \in (0, 1)$ and zero terminal reward $R_T \equiv 0$. Let $V_0^{*,T}(s)$ denote the optimal value at stage 0 with horizon T, and let $V^*(s)$ denote the optimal value of the corresponding infinite-horizon discounted MDP.

(a) Prove that $V_0^{*,T}(s) \leq V_0^{*,T+1}(s) \leq V^*(s)$ for all T and s.

(b) Prove that $V^*(s) - V_0^{*,T}(s) \leq \gamma^T R_{\max}/(1 - \gamma)$.

(c) Conclude that $V_0^{*,T}(s) \to V^*(s)$ as $T \to \infty$.

Exercise 8.6 (Secretary Problem Variant). $(***)$
In the classical secretary problem, the gambler observes only the *rank* of each offer relative to those seen so far, not its absolute value. Model this as a finite-horizon MDP: define the state space (what information is sufficient?), the action space, and the transition probabilities. Apply backward induction to derive the optimal stopping rule for $T = 4$ candidates and verify that the optimal strategy is to reject the first $\lfloor T/e \rfloor = 1$ candidate and then accept the next candidate that is the best seen so far.

9 Summary

- A finite-horizon MDP $(\mathcal{S}, \mathcal{A}, P, R, T)$ models sequential decision problems with a fixed terminal time T; actions are taken at stages $t = 0, \ldots, T - 1$.
- The value function $V_t^*(s)$ depends on both state and time, reflecting the fundamental non-stationarity of finite-horizon optimal behavior.

- The *principle of optimality* states that any tail of an optimal policy is itself optimal for the corresponding subproblem; this enables the recursive decomposition underlying dynamic programming.
- The finite-horizon Bellman optimality equations give a backward recursion: $V_t^*(s) = \max_a[R_t(s,a) + \sum_{s'} P_t(s' \mid s,a)V_{t+1}^*(s')]$ with boundary $V_T^* = R_T$.
- Backward induction computes the optimal policy and value function in $O(S^2AT)$ time, which is polynomial in all problem parameters.
- A deterministic Markov (but generally non-stationary) optimal policy always exists for finite-horizon MDPs with finite state and action spaces.
- The optimal stopping problem illustrates how optimal thresholds change with the horizon, a phenomenon absent in the infinite-horizon setting.
- As $T \to \infty$ with discounting, the finite-horizon optimal value converges to the infinite-horizon optimal value, bridging the two formulations.

10 Further Reading

- **Lesson 41** develops the infinite-horizon discounted setting: Bellman optimality equations, the contraction property of Bellman operators, and the existence of a single stationary optimal policy.
- **Lesson on Temporal-Difference Learning** extends the finite-horizon backward induction idea to the model-free setting, where the transition kernel is unknown.
- Puterman [2014, Chapter 4] covers finite-horizon MDPs with state-dependent action sets, randomized policies, and occupancy measures.
- Bertsekas [2012, Chapter 1] presents dynamic programming in the broader context of optimal control, including continuous state spaces and imperfect state information.
- The secretary problem and its many variants are surveyed in Ferguson (1989), "Who solved the secretary problem?", *Statistical Science*; the connection to online learning algorithms is developed in Kleinberg (2005).

References

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume II. Athena Scientific, 4th edition, 2012.

Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

Ronald A. Howard. Dynamic programming and markov processes. *MIT Press*, 1960.

Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2nd edition, 2014.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.